

Hernandez Gonzalez Mauricio

Hernández Aviña Karen Itzel

MPOO Grupo: 6

```
#####
#   TU TABLA FINAL   ·   ESTA ES LA CAPTURA QUE TIENES QUE ENTREGAR   #
#####
```

#	expresion	predijiste	salio	
1	enteroSinValor	0	0	ok
2	decimalSinValor	0.0	0.0	ok
3	logicoSinValor	false	false	ok
4	(int) letraSinValor	0	0	ok
5	textoSinValor	null	null	ok
6	7 / 2	3	3	ok
7	7.0 / 2	3.5	3.5	ok
8	(double) 7 / 2	3	3.5	FALLO
9	(double) (7 / 2)	3.5	3.0	FALLO
10	7.0 / 0	Infinity	Infinity	ok
11	0.0 / 0.0	Infinity	NaN	FALLO
12	7 % 2	1	1	ok
13	10 % 5	0	0	ok
14	-7 % 2	-1	-1	ok
15	7 % 2.5	2	2.0	ok
16	12345 % 10	5	5	ok
17	12345 / 10	1234	1234	ok
18	(int) 3.9	3	3	ok
19	(int) -3.9	-3	-3	ok
20	(char) 65	A	A	ok
21	(int) 'A'	65	65	ok
22	'A' + 1	66	66	ok
23	(char) ('A' + 1)	B	B	ok
24	(byte) 200	-56	-56	ok
25	5 > 3	true	true	ok
26	5 == 5.0	false	true	FALLO
27	0.1 + 0.2 == 0.3	false	false	ok
28	true && false	false	false	ok
29	true false	true	true	ok
30	vidasRestantes > 0 && 10 / vida~	false	false	ok
31	0.1 + 0.2	0.300000000000000~	0.30000000~	ok
32	Integer.MAX_VALUE + 1	-2147483648	-2147483648	ok
33	5 + 3	8	8	ok
34	"5" + 3	53	53	ok
35	1 + 2 + "3"	33	33	ok
36	"1" + 2 + 3	15	123	FALLO

```
-----
```

Lista de las preguntas fallidas

ACERTASTE 31 DE 36

ESTAS SON LAS QUE TIENES QUE EXPLICAR EN TU DOCUMENTO:

- `(double) 7 / 2`
creíste que daba: 3
en realidad da: 3.5
por que me equivoque: _____
- `(double) (7 / 2)`
creíste que daba: 3.5
en realidad da: 3.0
por que me equivoque: _____
- `0.0 / 0.0`
creíste que daba: Infinity
en realidad da: NaN
por que me equivoque: _____
- `5 == 5.0`
creíste que daba: false
en realidad da: true
por que me equivoque: _____
- `"1" + 2 + 3`
creíste que daba: 15
en realidad da: 123
por que me equivoque: _____

Explicaciones de las preguntas erróneas.

(double) 7 / 2 -> Nosotros pensamos que el casting marcado como (double) redondea el resultado a el valor entero más cercano, siendo el valor más cercano el 3. Sin embargo no contábamos con que el casting (double) altera el resultado final para que aparezca con todo y decimal.

(double) (7 / 2) -> En esta pregunta nosotros consideramos que los paréntesis no alteraban el casting, sin embargo pudimos notar que al estar encerrada la operación, el casting no se puede aplicar.

0.0 / 0.0 -> Debido a que ambos números de la operación son un tipo de dato double, Java al tratar con este tipo de datos implementa el estándar **IEEE 754**, que reserva ciertos patrones de bits específicos para referirse a un *Infinity* o *NaN*. Por lo que para operaciones indeterminadas como **0.0/0.0** Java

implementa lo que es el término de NaN (*Not a number o No es un número*).

5 == 5.0 -> En esta pregunta creímos que el tipo de dato de cada valor afectaba a la comparación, siendo que el 5 es un dato tipo int y que el 5.0 es un dato doble. Al parecer el tipo de dato no afecta cuando se hace una comparación con los signos ==.

"1" + 2 + 3 -> Aquí lo que pudimos notar fue la lectura de la máquina va de izquierda a derecha, donde debido a que el "1" es el primer término leído concatena y transforma en texto a los otros dos datos 2+3, y que los símbolos + son utilizados para juntar dichos valores convertidos en texto. Dando como resultado el texto "123".

Bloque 7. Código de operaciones.

```
=====
BLOQUE 7 · AHORA TU (aquí escribes código, no predicciones)
=====
7.1 = 2.5
7.2 = false
7.3 = 9
7.4 = true
7.5 = false
(sí todavía no escribes nada aquí, no se imprime nada: es normal)
```

Impresión del código del bloque 7

Hernandez Gonzalez Mauricio

Hernández Aviña Karen Itzel

MPOO Grupo: 6

```
//Código para el bloque 7
public void bloque7() {
    System.out.println();
    System.out.println("=====");
    System.out.println(" BLOQUE 7 - AHORA TU (aquí escribes código, no predicciones)");
    System.out.println("=====");

    //7.1 Que imprima exactamente 2.5, partiendo de los números 5 y 2.
    // Pista: si los dos quedan enteros nunca te dará 2.5. Haz que uno
    // sea decimal, con un casting o escribiéndolo como 2.0
    //Si dividimos un número entero entre un double, todo se hace double
    System.out.println("7.1 = " + (5/2.0));

    //7.2 Con %, una expresión que diga si 47 es par. Debe imprimir false
    //Usamos el operador % ya que sabemos que si el residuo de un número es 0, el número es primo
    System.out.println("7.2 = " + (47 % 2 == 0));

    //7.3 Un casting que convierta 9.99 en entero.
    // En el comentario escribe cuánto se perdió y por qué
    //En este cast explícito se corta el número a 9, se pierden dos decimales .99
    System.out.println("7.3 = " + (int) 9.99);

    //7.4 Comparar 0.1 + 0.2 contra 0.3 SIN usar ==, y que imprima true.
    // Pista: restalos y pregunta si la diferencia es menor que
    // 0.000001. Para que la resta nunca salga negativa usa
    // Math.abs( ... ), que devuelve el valor absoluto
    //Si hacemos una suma en decimales nos da un valor diferente a 0.3, al restar y usar el valor absoluto
    //se obtiene un valor cercano a 0, pero no negativo, por eso al comparar es true.
    System.out.println("7.4 = " + (Math.abs((0.1+0.2)-0.3) < 0.000001));

    //7.5 Un && que aproveche el corto circuito para NO dividir entre cero.
    // Declara un int en 0 con un nombre que diga que es, por ejemplo
    // divisor, y usalo en la condición. Que no truee
    //En este caso al evaluar la primera expresión 0=0 por lo que ya no entra a &&, ya que ambas se tienen que cumplir
    int divisor = 0;
    System.out.println("7.5 = " + (divisor != 0 && (10/divisor) > 0));

    //System.out.println(" (si todavía no escribes nada aquí, no se imprime nada: es normal)");
}
```

Código del bloque 7.